

A **DaemonSet** in Kubernetes (K8s) ensures that a copy of a specific pod runs on *all* (or a selected subset of) nodes in a cluster. It's a powerful resource for deploying background or system-level services that must be present on each node, like log collectors, monitoring agents, or networking daemons.

Core Concepts of DaemonSets

1. What It Does

A DaemonSet manages pods that run a copy of a daemon on every node. When:

- A new node is added to the cluster, the DaemonSet automatically schedules the pod on that node.
- A node is removed, the DaemonSet cleans up the pods.
- You delete the DaemonSet, Kubernetes deletes all the pods it created.

2. Use Cases

Common uses include:

- **Log collection** (e.g., Fluentd, Logstash)
 - **Monitoring** (e.g., Prometheus Node Exporter, Datadog Agent)
 - **Networking** (e.g., Calico, Weave Net agents)
 - **Security** (e.g., Falco)
-

How It Works

- **Controller Loop:** The DaemonSet controller watches for node additions and deletions.
 - **Pod Scheduling:** It directly schedules pods on the nodes, bypassing the default scheduler.
 - **Selector:** Matches the pod template using `.spec.selector` and `.spec.template.metadata.labels`.
-

Advanced Features

Tolerations and Node Selectors

You can constrain DaemonSets to run only on specific nodes.

```
spec:
  template:
    spec:
      nodeSelector:
        node-role.kubernetes.io/worker: ""
      tolerations:
        - key: "node-role.kubernetes.io/master"
          operator: "Exists"
          effect: "NoSchedule"
```

Running on Select Nodes

Use `nodeAffinity`, `nodeSelector`, or `labels` to target only specific subsets.

Rolling Updates

Modern DaemonSets (in `apps/v1`) support rolling updates and update strategies:

updateStrategy:
 type: RollingUpdate
 rollingUpdate:
 maxUnavailable: 1

DaemonSet vs Other Controllers

Controller	Runs Everywhere?	Use for Background Tasks?	Handles Node Events?
Deployment	No	Yes	No
StatefulSet	No	Sometimes	No
Job/CronJob	No	Yes (One-off or timed)	No
DaemonSet	Yes	Yes	Yes

Best Practices

- Use **labels** and **node selectors** to avoid scheduling on undesired nodes.
- Avoid resource-heavy daemons unless necessary, as they scale with the number of nodes.
- Use **updateStrategy** to ensure zero downtime during rolling updates.
- Monitor with tools like Prometheus to track pod health across nodes.

Let me know if you'd like [visual diagrams](#), [real-world examples](#), or a [comparison with Deployments](#).